

Kafka in Python — All Code Together

Unit IV · Producers + Consumers · Python versions of every example in the textbook

The textbook (Chapters 3 and 4) shows **Java**. Below is the same logic in **Python**. Most use `confluent-kafka` (the library behind the CIE solutions' producer); the two exam programs also have `kafka-python` versions, as used in your CIE-1 / CIE-2 2022 answer keys. Blue chips open the Java original in the textbook.

```
pip install confluent-kafka      # producer / consumer (librdkafka based)
pip install "confluent-kafka[avro]" # + Avro & Schema Registry
pip install kafka-python         # simpler pure-python client (used in CIE keys)
```

REMEMBER Index

- **Producers:** P1 basic config · P2 fire-and-forget · P3 synchronous · P4 async + callback · P5 reliable + high-throughput (EXAM) · P6 custom serializer · P7 Avro + Schema Registry · P8 custom partitioner · P9 headers · P10 interceptor-style counter · P11 JSON producer (`kafka-python`, EXAM)
- **Consumers:** C1 basic poll loop · C2 auto commit · C3 commit sync · C4 commit async · C5 async + sync combined · C6 commit specific offsets · C7 rebalance listener · C8 seek / offsets by time · C9 clean exit · C10 custom deserializer · C11 Avro deserializer · C12 standalone consumer (assign) · C13 assignor + static membership · C14 JSON consumer (`kafka-python`, EXAM) · C15 one consumer per thread

★ IMPORTANT Python ↔ Java name map

- Java `bootstrap.servers` etc. are the **same config keys** in a dict. `send()` → `produce()`; `Callback.onCompletion` → `on_delivery(err, msg)`; `Future.get()` → `flush()`.
- In `confluent-kafka` the callback fires inside `poll()` / `flush()`, so call `producer.poll(0)` regularly.
- Consumers: `poll(timeout)` returns **one message** (or `None`) in `confluent-kafka`; check `msg.error()` before using it.

Part 1 — Producer Code

Chapter 3 equivalents

P1. Create a producer (mandatory properties)

Java original: [→ Constructing · Part 2 p.5](#)

Mandatory: **`bootstrap.servers`**. In Python, serialization is done by you (`str` → `bytes`) or by a `SerializingProducer`; there is no `key.serializer` class setting for the plain `Producer`.

```
from confluent_kafka import Producer

conf = {"bootstrap.servers": "broker1:9092,broker2:9092"} # list >= 2 brokers
producer = Producer(conf)
```

P2. Fire-and-forget

Send and ignore the result (may lose messages silently). Java original: [→ Fire & forget · Part 2 p.8](#)

```
# topic, key, value (strings are encoded to bytes automatically)
producer.produce("CustomerCountry", key="Precision Products", value="France")
producer.poll(0)      # let the client do background work
producer.flush()      # make sure it leaves the buffer before exit
```

P3. Synchronous send

Waits for the broker reply (slow — rarely used in production). Java original: [→ Sync send · Part 2 p.9](#)

Python has no `send().get()`; the equivalent is to **`flush()`** after every record and check the callback result.

```

result = {}

def on_delivery(err, msg):
    result["err"], result["msg"] = err, msg

producer.produce("CustomerCountry", key="Precision Products", value="France",
                 callback=on_delivery)
producer.flush() # blocks until broker replies (or fails)

if result["err"] is not None:
    print("Send failed:", result["err"])
else:
    m = result["msg"]
    print(f"OK -> {m.topic()} [{m.partition()}] @ offset {m.offset()}")

```

P4. Asynchronous send with callback

Recommended: high throughput + error handling. Java original: [→ Async + callback · Part 2 p.11](#)

```

def on_delivery(err, msg):
    # = Callback.onCompletion(metadata, exception)
    if err is not None:
        print("Delivery failed:", err) # log / write to an errors file
    else:
        print(f"Delivered to {msg.topic()} [{msg.partition()}] offset {msg.offset()}")

for i in range(100):
    producer.produce("CustomerCountry", key=str(i), value="USA", callback=on_delivery)
    producer.poll(0) # serve callbacks without blocking
producer.flush()

```

P5. Reliable + high-throughput producer (no duplicates)

EXAM — CIE-1 2022: "high throughput and no duplicate orders". Java original: [→ Config · Part 2 p.12](#) [→ idempotence · Part 2 p.22](#)

acks=all + idempotence → no loss / no duplicates; linger.ms + batch.size + compression → throughput; key → per-key ordering.

```

from confluent_kafka import Producer
import json

conf = {
    "bootstrap.servers": "localhost:9092",
    "client.id": "order-service", # shows up in logs / quotas
    "acks": "all", # all in-sync replicas
    "enable.idempotence": True, # sequence numbers, no duplicates
    "delivery.timeout.ms": 120000, # keep retrying ~2 min (leader election)
    "linger.ms": 20, # wait to fill batches
    "batch.size": 65536, # bytes per batch
    "compression.type": "snappy",
    "max.in.flight.requests.per.connection": 5,
}
producer = Producer(conf)
TOPIC = "orders"

def on_delivery(err, msg):
    if err: print("FAILED:", err)
    else: print(f"{msg.topic()} [{msg.partition()}] @ {msg.offset()}")

order = {"order_id": 101, "item": "laptop", "qty": 1}
producer.produce(TOPIC, key=str(order["order_id"]), # same key -> same partition
                 value=json.dumps(order), callback=on_delivery)
producer.poll(0)
producer.flush()

```

P6. Custom serializer (Customer: id + name)

Java's CustomerSerializer: 4-byte id, 4-byte name length, then UTF-8 name. Java original: [→ Custom serializer · Part 2 p.24](#)

Fragile: adding a field breaks old data — prefer JSON / Avro / Protobuf (see P7).

```
import struct

class Customer:
    def __init__(self, cid, name):
        self.id, self.name = cid, name

def serialize_customer(c: Customer) -> bytes:
    if c is None:
        return None
    name = (c.name or "").encode("utf-8")
    # ">ii" = big-endian two 4-byte ints, like ByteBuffer.putInt()
    return struct.pack(">ii", c.id, len(name)) + name

producer.produce("customerCountries", key="Banana",
                 value=serialize_customer(Customer(7, "Banana Inc")))
producer.flush()
```

P7. Avro serializer with Schema Registry

Schema stored in the registry; only the schema id travels in each record. Java original: [→ Avro producer · Part 2 p.30](#)

```
from confluent_kafka import SerializingProducer
from confluent_kafka.schema_registry import SchemaRegistryClient
from confluent_kafka.schema_registry.avro import AvroSerializer
from confluent_kafka.serialization import StringSerializer

schema_str = """
{"namespace": "customerManagement.avro", "type": "record", "name": "Customer",
 "fields": [{"name": "id", "type": "int"},
             {"name": "name", "type": "string"},
             {"name": "email", "type": ["null", "string"], "default": null}]}
"""

registry = SchemaRegistryClient({"url": "http://localhost:8081"}) # schema.registry.url
avro_ser = AvroSerializer(registry, schema_str, lambda obj, ctx: obj) # dict -> Avro

producer = SerializingProducer({
    "bootstrap.servers": "localhost:9092",
    "key.serializer": StringSerializer("utf_8"),
    "value.serializer": avro_ser,
})

for n in range(3):
    customer = {"id": n, "name": f"exampleCustomer{n}", "email": f"example{n}@example.com"}
    producer.produce(topic="customerContacts", key=customer["name"], value=customer)
producer.flush()
```

P8. Custom partitioner ("Banana" gets its own partition)

The Python client lets you pick the partition explicitly. Java original: [→ Custom partitioner · Part 2 p.35](#)

Rule: "Banana" → last partition; every other key hashed over the remaining partitions. (Java version implements the Partitioner interface.)

```
import zlib

def banana_partition(topic, key: str) -> int:
    n = len(producer.list_topics(topic, timeout=10).topics[topic].partitions)
    if key is None:
        raise ValueError("We expect all messages to have customer name as key")
    if key == "Banana":
        return n - 1 # always the last partition
    return zlib.crc32(key.encode()) % (n - 1) # hash over the rest

key = "Acme"
producer.produce("customerCountries", key=key, value="USA",
                 partition=banana_partition("customerCountries", key))
producer.flush()
```

P9. Record headers

Metadata (lineage, routing, tracing) without touching key/value. Java original: [→ Headers · Part 2 p.37](#)

```
producer.produce("CustomerCountry", key="Precision Products", value="France",
                 headers=[("privacy-level", b"YOLO"), ("source", b"web-shop")])
producer.flush()
```

P10. Interceptor-style counter (onSend / onAcknowledgement)

Python has no interceptor plug-in; wrap produce() and the callback. Java original: [→ Interceptors · Part 2 p.37](#)

```
import threading, time
from confluent_kafka import Producer

class CountingProducer:
    def __init__(self, conf, window_s=10):
        self.p = Producer(conf)
        self.sent = self.acked = 0
        self.lock = threading.Lock()
        threading.Thread(target=self._report, args=(window_s,), daemon=True).start()

    def _ack(self, err, msg):          # like onAcknowledgement()
        with self.lock: self.acked += 1

    def produce(self, topic, **kw):    # like onSend(): count before sending
        with self.lock: self.sent += 1
        self.p.produce(topic, callback=self._ack, **kw)
        self.p.poll(0)

    def _report(self, window):
        while True:
            time.sleep(window)
            with self.lock:
                print("sent:", self.sent, "acked:", self.acked)
                self.sent = self.acked = 0

cp = CountingProducer({"bootstrap.servers": "localhost:9092"})
cp.produce("interceptor-test", value="hello")
cp.p.flush()
```

P11. JSON producer with kafka-python (orders_topic)

EXAM — CIE-2 2022: "Implement Kafka producer and consumer for the given scenario". Java original: [→ Sending · Part 2 p.8](#)

```
from kafka import KafkaProducer
import json

producer = KafkaProducer(
    bootstrap_servers="localhost:9092",
    value_serializer=lambda v: json.dumps(v).encode("utf-8"),  # dict -> bytes
    acks="all",
    retries=5,
)
order_data = {"order_id": 1, "customer": "Asha", "amount": 2499.0}

future = producer.send("orders_topic", order_data)  # asynchronous
meta = future.get(timeout=10)                       # optional: synchronous wait
producer.flush()
print(f"Order {order_data['order_id']} sent to Kafka! partition={meta.partition} offset={meta.offset}")
```

Part 2 — Consumer Code

Chapter 4 equivalents

C1. Basic consumer + poll loop

Creating, subscribing, polling. Java original: [→ Poll loop · Part 2 p.60](#)

Mandatory: bootstrap.servers + group.id. Deserialization is done by you (msg.value().decode()) or by a DeserializingConsumer.

```

from confluent_kafka import Consumer

conf = {
    "bootstrap.servers": "broker1:9092,broker2:9092",
    "group.id": "CountryCounter",
    "auto.offset.reset": "earliest",
}
consumer = Consumer(conf)
consumer.subscribe(["customerCountries"])      # or regex: ["^test.*"]

country_count = {}
try:
    while True:
        msg = consumer.poll(1.0)              # the poll loop - keep polling!
                                                # timeout in seconds
        if msg is None:
            continue
        if msg.error():
            print("Error:", msg.error()); continue
        country = msg.value().decode("utf-8")
        country_count[country] = country_count.get(country, 0) + 1
        print(f"topic={msg.topic()} partition={msg.partition()} "
              f"offset={msg.offset()} key={msg.key()} value={country}")
        print(country_count)
finally:
    consumer.close()

```

C2. Automatic commit

Every 5 s the offsets from the previous poll are committed. Java original: [→ Automatic commit · Part 2 p.75](#)
 Convenient but a crash re-processes up to 5 s of events (duplicates).

```

conf = {
    "bootstrap.servers": "localhost:9092",
    "group.id": "CountryCounter",
    "enable.auto.commit": True,                # default
    "auto.commit.interval.ms": 5000,           # default 5 s
}

```

C3. Manual commit — synchronous

Most reliable; blocks until the broker confirms; retries. Java original: [→ commitSync · Part 2 p.77](#)

```

conf = {"bootstrap.servers": "localhost:9092", "group.id": "g1",
        "enable.auto.commit": False}          # we commit ourselves
consumer = Consumer(conf); consumer.subscribe(["customerCountries"])

while True:
    msg = consumer.poll(1.0)
    if msg is None or msg.error():
        continue
    print(msg.topic(), msg.partition(), msg.offset(), msg.key(), msg.value())
    try:
        consumer.commit(message=msg, asynchronous=False) # commits offset+1 of this msg
    except Exception as e:
        print("commit failed", e)

```

C4. Manual commit — asynchronous

Faster; no retry (a late retry could overwrite a newer commit). Java original: [→ commitAsync · Part 2 p.78](#)

```

def on_commit(err, partitions):                # = OffsetCommitCallback.onComplete
    if err is not None:
        print("Commit failed for", partitions, err)

conf = {"bootstrap.servers": "localhost:9092", "group.id": "g1",
        "enable.auto.commit": False, "on_commit": on_commit}
consumer = Consumer(conf); consumer.subscribe(["customerCountries"])

while True:
    msg = consumer.poll(1.0)
    if msg is None or msg.error():
        continue
    print(msg.offset(), msg.key(), msg.value())
    consumer.commit(message=msg, asynchronous=True) # send and carry on

```

C5. Combine async (loop) + sync (shutdown)

Common production pattern. Java original: [→ Combined · Part 2 p.80](#)

```
closing = False
try:
    while not closing:
        msg = consumer.poll(1.0)
        if msg is None or msg.error():
            continue
        print(msg.offset(), msg.key(), msg.value())
        consumer.commit(message=msg, asynchronous=True)    # fast; next commit = retry
        consumer.commit(asynchronous=False)                # last commit must succeed
except Exception as e:
    print("Unexpected error", e)
finally:
    consumer.close()
```

C6. Commit specific offsets (every 1000 records)

Commit **offset + 1** = the next message to read. Java original: [→ Specific offsets · Part 2 p.81](#)

```
from confluent_kafka import TopicPartition

current_offsets = {}                                # (topic, partition) -> next offset
count = 0
while True:
    msg = consumer.poll(1.0)
    if msg is None or msg.error():
        continue
    print(msg.topic(), msg.partition(), msg.offset(), msg.key(), msg.value())
    current_offsets[msg.topic(), msg.partition()] = msg.offset() + 1
    if count % 1000 == 0:
        offsets = [TopicPartition(t, p, o) for (t, p), o in current_offsets.items()]
        consumer.commit(offsets=offsets, asynchronous=True)
    count += 1
```

C7. Rebalance listener (commit before losing partitions)

on_assign / on_revoke / on_lost = onPartitionsAssigned / Revoked / Lost. Java original: [→ Rebalance listeners · Part 2 p.83](#)

```
from confluent_kafka import TopicPartition

current_offsets = {}

def on_assign(consumer, partitions):
    print("Assigned:", partitions)                # load state / seek here

def on_revoke(consumer, partitions):
    print("Lost partitions in rebalance. Committing:", current_offsets)
    offsets = [TopicPartition(t, p, o) for (t, p), o in current_offsets.items()]
    if offsets:
        consumer.commit(offsets=offsets, asynchronous=False)    # sync before rebalance

def on_lost(consumer, partitions):
    print("Partitions lost (cooperative):", partitions)

consumer.subscribe(["customerCountries"],
                   on_assign=on_assign, on_revoke=on_revoke, on_lost=on_lost)

while True:
    msg = consumer.poll(1.0)
    if msg is None or msg.error():
        continue
    print(msg.topic(), msg.partition(), msg.offset(), msg.key(), msg.value())
    current_offsets[msg.topic(), msg.partition()] = msg.offset() + 1
```

C8. Seek: beginning, end, or one hour ago

Start reading from a different offset. Java original: [→ Seek · Part 2 p.87](#)

```

import time
from confluent_kafka import TopicPartition, OFFSET_BEGINNING, OFFSET_END

# beginning / end of every assigned partition
def seek_beginning(consumer, partitions):
    for p in partitions: p.offset = OFFSET_BEGINNING # or OFFSET_END
    consumer.assign(partitions)
consumer.subscribe(["customerCountries"], on_assign=seek_beginning)

# rewind to one hour ago (offsets_for_times uses a timestamp in ms)
one_hour_ago = int((time.time() - 3600) * 1000)
tps = [TopicPartition(p.topic, p.partition, one_hour_ago) for p in consumer.assignment()]
for tp in consumer.offsets_for_times(tps, timeout=10):
    if tp.offset >= 0:
        consumer.seek(tp) # next poll starts here

```

C9. Clean exit (wakeup / shutdown hook)

Stop the loop, then **close()** so offsets are committed and a rebalance happens immediately. Java original: [→ Exit · Part 2 p.88](#)

```

import signal

running = True
def shutdown(signum, frame): # = ShutdownHook calling consumer.wakeup()
    global running
    print("Starting exit...")
    running = False
signal.signal(signal.SIGINT, shutdown)
signal.signal(signal.SIGTERM, shutdown)

try:
    while running:
        msg = consumer.poll(10.0) # long timeout = low CPU on quiet topics
        print("waiting for data..." if msg is None else msg.value())
        if msg is not None and not msg.error():
            consumer.commit(message=msg, asynchronous=False)
finally:
    consumer.close() # commits offsets, leaves the group
    print("Closed consumer and we are done")

```

C10. Custom deserializer (Customer)

Mirror of P6; producer and consumer must agree on the format. Java original: [→ Custom deserializer · Part 2 p.92](#)

```

import struct

def deserialize_customer(data: bytes):
    if data is None:
        return None
    if len(data) < 8:
        raise ValueError("Size of data received by deserializer is shorter than expected")
    cid, name_size = struct.unpack(">ii", data[:8])
    name = data[8:8 + name_size].decode("utf-8")
    return {"id": cid, "name": name}

msg = consumer.poll(1.0)
if msg and not msg.error():
    c = deserialize_customer(msg.value())
    print("current customer Id:", c["id"], "and name:", c["name"])

```

C11. Avro deserializer with Schema Registry

Uses the schema id in the record to fetch the writer schema. Java original: [→ Avro deserializer · Part 2 p.94](#)

```

from confluent_kafka import DeserializingConsumer
from confluent_kafka.schema_registry import SchemaRegistryClient
from confluent_kafka.schema_registry.avro import AvroDeserializer
from confluent_kafka.serialization import StringDeserializer

registry = SchemaRegistryClient({"url": "http://localhost:8081"})
consumer = DeserializingConsumer({
    "bootstrap.servers": "localhost:9092",
    "group.id": "CountryCounter",
    "key.deserializer": StringDeserializer("utf_8"),
    "value.deserializer": AvroDeserializer(registry),      # schema comes from the registry
    "auto.offset.reset": "earliest",
})
consumer.subscribe(["customerContacts"])
while True:
    msg = consumer.poll(1.0)
    if msg is None or msg.error():
        continue
    print("Current customer name is:", msg.value()["name"]) # value() is a dict
    consumer.commit(message=msg, asynchronous=False)

```

C12. Standalone consumer (no group, no rebalance)

assign() the partitions yourself — do not subscribe(). Java original: [→ Standalone · Part 2 p.96](#)

Not notified when partitions are added: re-check list_topics() periodically or restart.

```

from confluent_kafka import Consumer, TopicPartition

consumer = Consumer({"bootstrap.servers": "localhost:9092",
                    "group.id": "standalone",           # still needed to commit offsets
                    "enable.auto.commit": False})
parts = consumer.list_topics("topic", timeout=10).topics["topic"].partitions
consumer.assign([TopicPartition("topic", p) for p in parts]) # all partitions of "topic"

while True:
    msg = consumer.poll(1.0)
    if msg is None or msg.error():
        continue
    print(f"topic={msg.topic()} partition={msg.partition()} offset={msg.offset()} "
          f"key={msg.key()} value={msg.value()}")
    consumer.commit(message=msg, asynchronous=False)

```

C13. Assignor + static membership + key timeouts

Config only. Java original: [→ Assignors · Part 2 p.69](#) [→ Static membership · Part 2 p.56](#)

Strategies: range (default), roundrobin, cooperative-sticky (incremental rebalance). In librdkafka the plain "sticky" is not offered separately.

```

conf = {
    "bootstrap.servers": "localhost:9092",
    "group.id": "sales_group",
    "partition.assignment.strategy": "cooperative-sticky", # or "range" / "roundrobin"
    "group.instance.id": "consumer-1",                    # static member: restart w/o rebalance
    "session.timeout.ms": 10000,                          # dead if no heartbeat for this long
    "heartbeat.interval.ms": 3000,                        # ~ 1/3 of session timeout
    "max.poll.interval.ms": 300000,                      # max gap between poll() calls
    "fetch.min.bytes": 1,
    "fetch.wait.max.ms": 500,                             # = fetch.max.wait.ms
    "max.partition.fetch.bytes": 1048576,
    "auto.offset.reset": "earliest",
    "enable.auto.commit": False,
}

```

C14. JSON consumer with kafka-python (sales_data)

EXAM — CIE-1 2022 & CIE-2 2022: auto-convert JSON into a Python dictionary. Java original: [→ Deserializers · Part 2 p.90](#)

```

from kafka import KafkaConsumer
import json

def json_deserializer(data):
    return json.loads(data.decode("utf-8"))

consumer = KafkaConsumer(
    "sales_data",
    bootstrap_servers="localhost:9092",
    group_id="sales_group",
    auto_offset_reset="earliest",
    enable_auto_commit=True,
    value_deserializer=json_deserializer,
)
print("Consumer is listening to 'sales_data' topic...")
for message in consumer:
    print(f"Topic: {message.topic}, Partition: {message.partition}, Offset: {message.offset}")
    print(f"Message Value (Deserialized): {message.value}")

```

C15. One consumer per thread (same group)

A consumer object is not thread-safe: never share one across threads. Java original: [→ Thread safety · Part 2 p.62](#)

```

import threading
from confluent_kafka import Consumer

def run_consumer(name, stop):
    c = Consumer({"bootstrap.servers": "localhost:9092", "group.id": "CountryCounter",
                  "auto.offset.reset": "earliest"})
    c.subscribe(["customerCountries"])
    try:
        while not stop.is_set():
            msg = c.poll(1.0)
            if msg and not msg.error():
                print(name, msg.partition(), msg.offset(), msg.value())
    finally:
        c.close()

stop = threading.Event()
threads = [threading.Thread(target=run_consumer, args=(f"C{i}", stop)) for i in range(3)]
for t in threads: t.start()
# ... later: stop.set(); [t.join() for t in threads]

```

EXAM TIP Before the exam

- Programs most likely to be asked: **P5 / P11** (producer) and **C14** (consumer). Be able to explain each config line.
- For any consumer program mention: group.id, deserializer, auto_offset_reset, commit strategy, and that the loop is the poll loop.
- These were written from the textbook's Java examples and the CIE answer keys; install the libraries and a local broker (localhost:9092) to run them.